

Agent Orchestration

Sergei Telegin

github.com/TeleginS/agent-orchestration

157 tests passed. QA still blocked the PR

AI agents: the execution loop and a real delivery pipeline

TrafficRulesApp

My iOS app for the Spanish driving theory exam

Review approved the change.
QA found problems in the tests themselves.

Sergei Telegin / Manychat meetup

Saved run reports. The pull request was still open when these screenshots were captured.

QA — Pass 1

Verdict:  **BLOCKED** on three in-scope findings.

Test suite

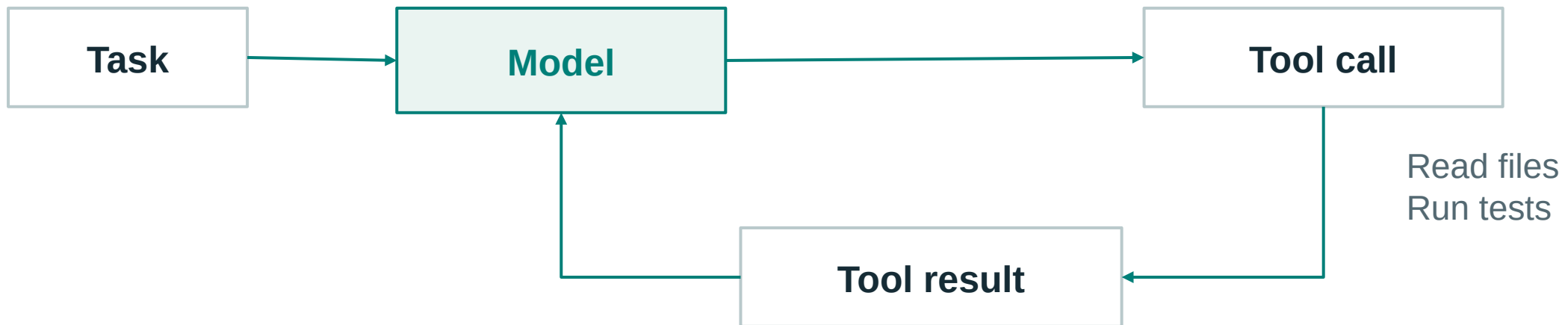
Four runs, two destinations. Counts from `xcrun xcrresulttool get test-results summary`, not console grep.

#	Destination	Mode	Result
1	iPhone 17	default (parallel)	157 passed / 0 failed / 0 skipped
2	iPhone 11 Pro Max	default (parallel)	157 / 0 / 0
3	iPhone 17	<code>-parallel-testing-enabled NO</code>	157 / 0 / 0
4	iPhone 17	<code>-only-testing the two new classes</code>	32 / 0 / 0

The agent loop

The model chooses actions. The harness executes allowed calls.

Task: add a regression test

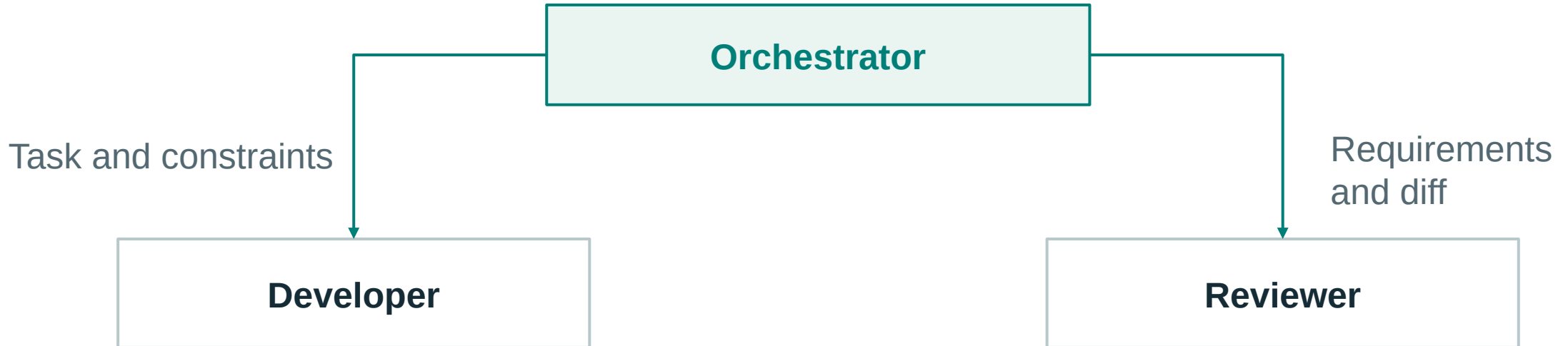


Harness

Context, permissions, error handling and stopping conditions

Delegation separates tasks and context

The reviewer receives requirements and the actual change



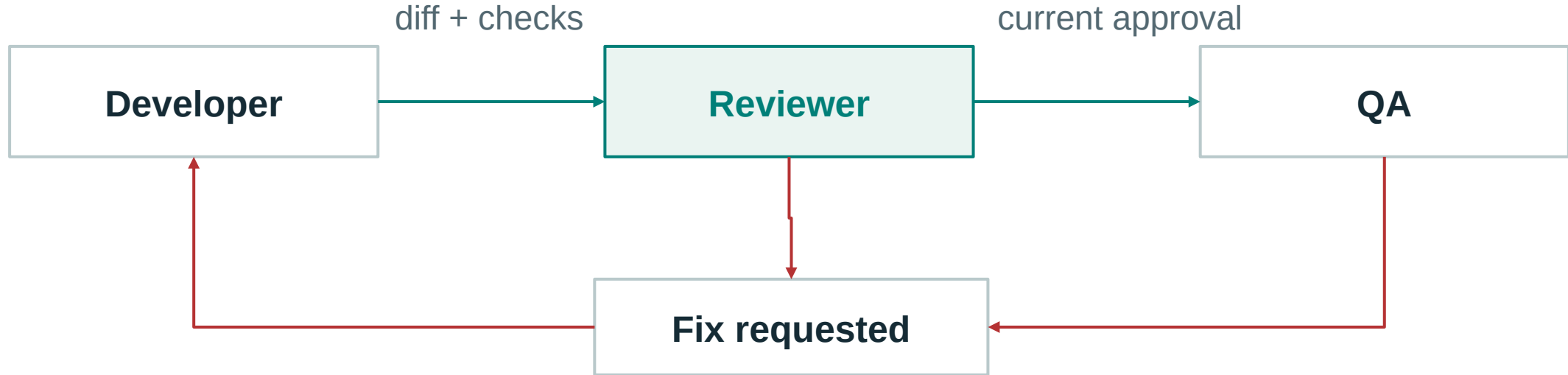
Implementation and check results

Findings or approval

Separate context improves focus. Shared assumptions can still produce shared errors.

Stage freedom and transition rules

A stage can choose its actions while the route requires evidence



Inside a stage

Choose files and an implementation

Between stages

Require evidence and bound retries

A prompt describes a rule. Runtime code can enforce it, but cannot guarantee a correct review.

My pipeline changed after earlier runs

A versioned rule connects the earlier case to the later run



Then review the new version before QA runs again

July: review and bug

[BUG] exitTest() double-counts session time into ProgressStore when it fires after natural completion #144 [✎](#)

26 August: rule 11

Verdict:  APPROVED

September: coverage run

```
185 + 11. **Every code change after QA begins must pass code review before QA can be considered green.** QA bug fixes go through ios-code-reviewer before qa-bug-tracker re-tests them.
```

Prompt-based routing. Current generic pipeline: stop after three unsuccessful iterations per loop.

Accumulated time or this call's contribution?

Five tests asserted an absolute total while sharing persistent storage

Illustrative example

Before	500 seconds
After	500 seconds

after > 0 **PASS**

after > before **FAIL**

The call added nothing.

Second review: approval

 **APPROVAL**

QA found the missing proof

-  [\[BUG\] Five terminal-path tests assert an absolute totalTestSeconds instead of a delta #199 \(medium\)](#) — five single-path tests assert `totalTestSeconds > 0` instead of a delta. Whenever the store is non-zero on entry the assertion is a tautology and cannot fail even if the accumulation line were deleted from the method under test. This is the half of PR [Cumulative all-time Progress statistics #143](#) invariant 1 the task exists to pin; the "not twice" half was written correctly with real snapshots, so the file is internally inconsistent. Not theoretical — it already happened in this task, and the suite still leaves `progress*` residue behind ( [\[BUG\] TestSession tests leave progress* keys in UserDefaults.standard #202](#)).

Paired fix: compare with the prior value and give each test its own isolated store.

The fix gets its own review

The same change passes review before QA runs again

Fix: delta assertions and isolated test storage

Fix QA findings on ProgressStore tests

Assert deltas, not absolute totals, on the five accumulation paths – the old `> 0` form was a tautology whenever the store was non-zero on entry.

Review 3

Code Review — Iteration 3 (review gate on the QA fixes)

Pipeline rule 11: every code change made after QA begins must pass review before QA re-tests. Scope: `e19273c..114cb54` — 2 files, +49/–57, test code only.

 **APPROVAL**

After isolation, before = 0.
The two fixes work together.

QA retest

QA — Pass 2 (re-test)

Verdict:  GREEN.

Run	Destination	Condition	Passed	Failed	Skipped
1	iPhone 17 (iOS 26.5)	baseline	157	0	0

157 passed in all four full runs

The cost of this run

32 new tests and a project profile in the same pull request

Measure	Recorded result
Subagent launches	9
Elapsed time	About 1 h 31 min
Subagent tokens	1,097,667

3 developer, 3 reviewer,
2 QA, 1 planner

Measured: subagent tokens

Step	Role	Tokens	Tool calls	Wall clock
1	task-planner	113,654	24	7m 10s
3	developer	159,422	59	24m 52s
4 · iter 1	code-reviewer	134,464	30	7m 57s
4 · fix	developer	74,767	19	4m 00s
4 · iter 2	code-reviewer	95,164	20	5m 41s
5	qa-tester	164,345	37	12m 31s
6a	developer	96,169	30	8m 32s
6b	code-reviewer	118,791	36	6m 43s
6c	qa-tester	140,891	47	13m 29s
Total	9 launches	1,097,667	302	1h 31m

Orchestrator tokens excluded. No comparable single-agent control run.

What I changed and what comes next

Observations from the run feed back into the process

Observation	Response
QA fixes change reviewed code	Another review before QA
Project facts drift	A shared profile, corrected from evidence
Text rules can be skipped	Next: enforce transitions in the runtime
Coordination consumes resources	Next: measure human intervention too

**Profile correction:
13 to 15 test files**

The active profile says "Tests: TrafficTestAppTests/ (13 test files)" — there are now 15, totalling 157 tests. Worth recording the count as a stated baseline so the next iteration can spot a real regression instead of chasing a grep artifact.

Current routing relies mainly on prompts. Runtime enforcement is a proposed improvement.

A first experiment in your own project

One task type

A checkable result for every stage

A saved record of each run

Findings, instruction version, costs and human intervention

Roles, project profiles and run reports

github.com/TeleginS/agent-orchestration